

Lenguaje Python

Cursada 2026

Tuplas y diccionarios

Trabajamos con listas

```
In [ ]: my_list = ["La tregua", "Camila", "La historia oficial",  
                 "Tango, no me dejes nunca", "El hijo de la novia",  
                 "Argentina, 1985,"]  
my_list
```

Dijimos que una lista es una **colección heterogénea** de datos. Esto significa que:

```
In [ ]: my_list.append(10)  
my_list
```

¿Qué otras operaciones podemos hacer con listas?

Slicing con listas

- Al igual que en el caso de las cadenas de caracteres, se puede obtener una porción de una lista usando el operador ":"

```
In [ ]: vowels = [ "a", "e", "i", "o", "u"]  
print(vowels[1:5:2])
```

- Si no se pone inicio o fin, se toma por defecto las posiciones de inicio y fin de la lista.

CONSIGNA: probar en casa con índices negativos.

Tuplas: otro tipo de secuencias en Python

Son colecciones heterogéneas de datos ordenados.

```
In [ ]: tuple0 = 1, 2  
tuple1 = (1, 2)
```

```
tuple2 = (1, ) # OJO con esto
tuple3 = ()
```

```
In [ ]: type(tuple2)
```

Observemos detalladamente el código del DESAFÍO 4 de la clase

Dada una lista de palabras, **usando list comprehension** generar otra lista con aquellos verbos en infinitivo.

IMPORTANTE: solo vamos a comprobar que terminen en "ar", "er" o "ir".

```
In [ ]: words = ["casa", "ir", "sol", "cantar", "correr"]
verbs = [pal for pal in words if pal.endswith(("ar", "er", "ir"))]
verbs
```

En este caso, estamos pasando una tupla como argumento al método **endswith**.

¿Cuál es la diferencia con las listas?

Veamos las siguientes situaciones.

Tuplas vs. listas

```
In [ ]: my_tuple = ("1_in_tuple", 2)
my_list = ["1_in_list", 2]
```

```
In [ ]: elem = my_tuple[0]
print(elem)
print(len(my_tuple))
```

- Se acceden a los elementos de igual manera: usando [] (empezando desde cero)
- La función **len** retorna la cantidad de elementos en ambos casos.

¿Entonces?

DIFERENCIA: las tuplas son INMUTABLES

- Su tamaño y los valores de las mismas NO pueden cambiar.

```
In [ ]: my_tuple = ("1_in_tuple", 2)
my_list = ["1_in_list", 2]
```

```
In [ ]: #my_list[0] = "uno"
my_list.append("algo")
my_list
```

```
In [ ]: my_tuple[0] = "uno" # Esto da error
        #my_tuple.append("algo") # Esto da error
        my_tuple
```

TypeError: 'tuple' object does not support item assignment

Tenemos que acostumbrarnos a leer los errores.

Obteniendo subtuplas

```
In [ ]: my_tuple = (1, 2, 3, "hola")
        print(my_tuple[1:4])
        new_tuple = ("nueva",) + my_tuple[:2]
        print(new_tuple)
```

¿Por qué da error este código?

```
In [ ]: new_tuple = ('nueva') + my_tuple[1:3]
        new_tuple
```

DESAFÍO 1

Volvemos a procesar las películas trabajadas en la clase.

Queremos saber:

- cuál fue la duración, en minutos, promedio; y
- **qué películas** duran más que el promedio, en minutos.

¿Qué diferencia hay con el desafío trabajado en clase?

- Deberíamos ingresar no sólo las duraciones, sino también los nombres de las películas.
- ¿Qué soluciones proponen?

¿Qué les parece esta solución?

```
In [ ]: name = input("Ingresa el nombre de una película (<FIN> para finalizar):")
        movies_list = []
        while name != "FIN":
            duration = int(input(f"Ingresa la duración de la película {name}: "))
            movies_list.append((name, duration))
            name = input("Ingresa el nombre de una película (<FIN> para finalizar)")
        movies_list
```

- ¿Qué estructura de datos estoy usando? ¿Podría usar una tupla?

- Hay algo mejor...

Diccionarios en Python

Un diccionario es un conjunto **ordenado** de pares de datos: **clave:valor**.

- Se definen con {}.

```
In [ ]: movies = {"La tregua": 108,  
                  "Camila": 105,  
                  "La historia oficial": 112,  
                  "Tango, no me dejes nunca": 115,  
                  "El hijo de la novia": 123,  
                  "El secreto de sus ojos": 129,  
                  "Relatos salvajes": 122,  
                  "Argentina, 1985": 140  
                  }
```

Las claves deben ser únicas e inmutables

- Las **claves** pueden ser cualquier tipo **inmutable**.
 - Las cadenas y números siempre pueden ser claves.
 - Las tuplas se pueden usar sólo si no tienen objetos mutables.

CONSIGNA: **probar en casa** cuáles de las siguientes definiciones de diccionarios dan error y analizar por qué.

```
In [ ]: my_dic = {"uno": 1}  
#my_dic = {1: "uno"}  
#my_dic = {[1,2]: "lista"}  
#my_dic = {(1,2): "tupla"}  
#my_dic = {[1], 2): "tupla"}  
my_dic
```

¿Cómo accedemos a los elementos?

- Al igual que las listas y tuplas, se accede usando [] pero en vez de un índice que representa la posición, **usamos la clave**.
- Es un error extraer un valor usando una clave no existente.

```
In [ ]: movies["La historia oficial"]
```

Otra forma:

```
In [ ]: movies.get("La historia oficial")
```

¿Cómo agregamos elementos?

- Si se usa una clave que ya está en uso para guardar un valor, el valor que estaba asociado con esa clave se pierde, si no está la clave, se agrega.

```
In [ ]: months = {"enero": 31, "febrero": 28, "marzo": 31}
months
```

```
In [ ]: months["febrero"] = 29
months["abril"] = 30
months
```

Volviendo al desafío de las películas ...

- Nos falta saber cómo definir un diccionario vacío para luego ir agregando los valores.

```
In [ ]: name = input("Ingresa el nombre de una película (<FIN> para finalizar):")
dic_movies = {}
while name != "FIN":
    duration = int(input(f"Ingresa la duración de la película {name}: "))
    dic_movies[name] = duration
    name = input("Ingresa el nombre de una película (<FIN> para finalizar):")
dic_movies
```

¿Cómo recorremos un diccionario?

```
In [ ]: music = {"rock": ["Riff", "La Renga", "La Torre"],
                "blues": ["La Mississippi", "Memphis"]}
```

```
In [ ]: for elem in music:
        print(elem)
```

¿A qué referencia la variable **elem**?

Si queremos mostrar los valores:

```
In [ ]: #music = {"rock": ["Riff", "La Renga", "La Torre"],
#           "blues": ["La Mississippi", "Memphis"]}

for elem in music:
    print(music[elem])
```

CONSIGNA: probar en casa

- Existen algunos métodos útiles. ¿Para qué creen que sirven?
- ¿Cómo procesamos los datos obtenidos por el método items?

```
In [ ]: music_keys = music.keys()
music_values = music.values()
music_items = music.items()
```

```
In [ ]: ##NO SE MUESTRA

for elem in music_values:
    print(elem)
```

```
In [ ]: ##NO SE MUESTRA

music_keys
```

```
In [ ]: ##NO SE MUESTRA

type(music_keys)
```

El operador `in` en diccionarios

¿Verifica en las claves o los valores?

```
In [ ]: music = {"rock": ["Riff", "La Renga", "La Torre"],
                "blues": ["La Mississippi", "Memphis"]}

"rock" in music
```

Observemos este código

```
In [ ]: months = {"enero": 31, "febrero": 28, "marzo": 31}
months1 = months
months2 = months.copy()
print(id(months))
print(id(months1))
print(id(months2))
```

¿Qué significa?

```
In [ ]: #months1["abril"] = 30
months2["abril"] = 43
months
```

¡Atención con las copias!

CONSIGNA: analizar qué sucede en estas situaciones.

```
In [ ]: claudia_notes = {"Python_plus": [10, 40, 20],
                        "Práctica": [10, 10]}

claudia_notes
```

```
In [ ]: sofia_notes = claudia_notes.copy()
sofia_notes
```

```
In [ ]: claudia_notes["Python_plus"].append(0)
claudia_notes
```

```
In [ ]: sofia_notes
```

CONSIGNA: probar en casa

- **del**: permite borrar un par clave:valor
- **clear()**: permite borrar todo

```
In [ ]: # Probamos del y clear
```

Otra forma de crear diccionarios

- Podemos usar **dict()**.
- Se denomina **constructor** y crea un diccionario directamente desde una secuencia de pares clave-valor.

```
In [ ]: months = dict([("enero", 31), ("febrero", 28), ("marzo", 31)])
months
```

- ¿Qué tipos de datos se pueden usar para la secuencia?
- ¿Y para los pares clave-valor?

Definición por comprensión

```
In [ ]: dict([(num, num**2) for num in (2, 4, 6)])
```

```
In [ ]: import string

ascii_numbers = dict([n, ord(n)] for n in string.digits)
ascii_numbers
```

```
In [ ]: squares = {num: num**2 for num in range(1, 10)}
squares
```

Otras funciones útiles

Observemos qué realizan los siguientes códigos:

```
In [ ]: students_notes = {"Janis Joplin": 10, "Bob Marley": 8, "Bruce Springsteen": 9}
```

```
In [ ]: all(students_notes.values())
```

```
In [ ]: min(students_notes.values())
```

```
In [ ]: len(students_notes)
```

